Notes for ECE 46800 - Introduction to Compilers and Translation Engineering

Zeke Ulrich

August 26, 2026

Contents

Course Description 1

Compiler Basics 2

Phases of a Compiler 3

Course Description

The design and construction of compilers and other translators. Topics include compilation goals, organization of a translator, grammars and languages, symbol tables, lexical analysis, syntax analysis (parsing), error handling, intermediate and final code generation, assemblers, interpreters, and an introduction to optimization. Emphasis is on engineering a compiler or interpreter for a small programming language - typically a C or Pascal subset. Projects involved the stepwise implementation (and documentation) of such a system.

Compiler Basics

A *compiler* is commonly understood as a program that converts a high-level language to a low-level language executable by hardware. However, in general a compiler translates one representation of a program to another, and perhaps does some analysis along the way.

All translation is transformation. A Greek-to-English translation of the Odyssey will preserve the meaning while reading as close as possible to the original. Code translation is the same. There are many possible translations of a given program to another language, with different properties and optimizations. Two translations that accomplish the same task may run in different times or use different amounts of memory.

Compilers save poor programmers from writing directly in assembly. The many flavours of high-to-low compilers enable us to write in one language and compile to many different architectures, instead of needing to write the same program in x86 for x86 machines, RISC-V for RISC-V chips, and so on.

Phases of a Compiler

The phases are:

- scanner
- parser
- semantic actions
- optimizer
- code generator

The compiler starts by seeing only text, `'i' 'f' ' ' '(' ....` The *scanner* phase converts raw text to a series of tokens, one for each "word" in the program, `if ( ID(a) OP(<) LIT(4) ) ....`

The *parser* then takes this string of tokens and coverts it to a *parse tree* or *abstract syntax tree* which captures the structure of the code, "this is an if-then, in the first case follow this code path...".

Syntax tells what symbols, in what order, are legal in a language. Semantics is the meaning. The phrase "gorilla yip huffly huffly" is legal syntax in English, but has no semantic meaning. Semantic actions are taken by compiler based on the semantics of program statements, including building a *symbol table* and generating *intermediate representations*.

Symbol tables associate each identifier or symbol (variables, constants, procedures, functions, etc.) in a program's code with the necessary information for its functioning. This may include type and scope of a variable, return type for a function, etc.

Intermediate representation is a low level representation of the program that is not machine specific (and therefore does not reference registers), such as three-address code.

The *optimizer* transforms code to make it more efficient. There are many optimizations to make, especially for code produced by students.

The *code generator* generates assembly from the intermediate represention.

The scanner, parser, and semantic actions together are called the front end of a compiler. IR-level optimizations and code generation are the back end. You can make multiple front ends for a back end, and multiple back ends for a front end.